Shaun the Sheep

Interactive Website Hero • Google AI Studio Prompt

6-second video support + performance optimization built in

Copy the prompt below into Google AI Studio along with the video, logo, controller icon, and two reference images.

ROLE & GOAL

Act as a senior creative frontend engineer specializing in high-performance interactive web experiences. Build a complete, polished, responsive hero section for a premium interactive “Shaun the Sheep” game website using React, TypeScript, Vite, and Tailwind CSS. The result must be visually faithful to the supplied references, smooth on typical desktop hardware, and maintainable.

ATTACHED ASSETS

I am attaching:
1. Shaun the Sheep background video (6 seconds)
2. Transparent Shaun the Sheep logo
3. Game/controller icon
4. FIRST reference image = initial/collapsed state
5. SECOND reference image = expanded state

Use the reference images ONLY for layout, composition, positioning, and visual direction. Never use them as page backgrounds. Use the supplied video as the actual animated Shaun background; do not substitute a screenshot, still image, or generated placeholder.

CORE EXPERIENCE

The 6-second attached video is the real hero background and contains Shaun’s movement. The interaction is a direct, absolute-position vertical scrubber controlled by the pointer—not normal video playback and not accumulated mouse distance.

Initial state: progress = 0, video starts at time 0, Shaun is in the close-up/collapsed position from reference 1, and the colorful elements are in their compact start positions.
Pointer moves upward: progress increases, video moves forward, Shaun rises/reveals more of his body, and colorful elements spread toward the positions in reference 2 as if Shaun physically pushes them outward.
Pointer moves downward: progress decreases, video reverses, Shaun returns toward the close-up, and elements return toward their compact positions.
Pointer stops: freeze the video at its current frame and hold all elements at the matching positions. No animation may continue independently after the pointer stops.

The full scrub range must be reachable: bottom interaction boundary = 0%; top/header interaction boundary = 100%. Reaching the top boundary must reach the final frame of the full 6-second clip.

VIDEO-TO-POINTER MAPPING

Use the pointer's absolute Y coordinate. Do not use deltaY, accumulated movement, wheel scrolling, or elapsed time to determine progress.

Define configurable interaction boundaries:
- **topBoundary:** the top edge of the active header/interaction area (or a clearly defined small offset below the viewport top).
- **bottomBoundary:** the bottom edge of the hero interaction area.

Calculate and clamp normalized progress:
progress = clamp((bottomBoundary - pointerY) / (bottomBoundary - topBoundary), 0, 1)
Then calculate:
targetTime = progress × video.duration

Use the loaded **video.duration** as the source of truth; the supplied clip is expected to be 6 seconds, but do not hardcode 6 if the actual metadata differs. Ensure the video metadata is loaded before calculating targetTime. Pointer at bottom = 0 seconds; pointer at top = full duration. Pointer stationary = unchanged progress and frozen frame. Returning to either boundary must reliably return to the corresponding endpoint. Keep the same single normalized progress as the source for video time and floating-element positions.

VIDEO IMPLEMENTATION & RELIABILITY

Implement a real HTML element behind the interface:
- position: fixed; inset: 0; width: 100%; height: 100%; object-fit: cover
- muted and playsInline; preload="auto"; appropriate accessible fallback text
- place it behind all UI using deliberate z-index layers
- do not autoplay, loop, or call video.play() under any circumstance
- initialize to currentTime = 0 once metadata is available
- handle loadedmetadata, canplay, waiting, seeking, seeked, and error states safely
- verify the asset exists in the project and the Vite-compatible source path resolves correctly
- preserve the supplied video's appearance and aspect ratio with object-fit: cover

If the video asset is missing or the path is invalid, report that the asset must be added or the path corrected. Do not silently replace it with a static image or placeholder. Keep UI visible while the video loads, without triggering automatic playback.

PERFORMANCE & SMOOTHNESS — IMPORTANT

Build performance optimization into the first implementation so the experience does not lag, stutter, freeze, or get stuck during pointer movement. Profile the interaction mentally and avoid unnecessary work on every pointer event.

Pointer and rendering:
- Use pointer events and store the latest pointerY/progress in refs; do not update React state on every pointer movement.
- Use one requestAnimationFrame loop at most, scheduled only when new input or pending work exists; cancel/stop it when idle.
- Keep interaction logic centralized in a reusable controller and avoid duplicate listeners or animation loops.
- Apply element movement through CSS transforms/CSS custom properties or refs, using translate/rotate/scale; avoid layout-triggering properties such as top, left, width, and height during animation.
- Avoid expensive blur, large animated shadows, filters, repeated measurements, forced synchronous layout, and unnecessary DOM reads/writes. Measure boundaries on resize/orientation change, not every frame.
- Use will-change: transform only on elements that actually animate, and avoid excessive GPU layers.

Video seeking:
- Never call video.play() and never allow time-based playback to continue after input stops.
- Avoid assigning video.currentTime for every raw pointer event. Coalesce input and issue seeks from the animation frame scheduler.
- Maintain a latest targetTime and allow at most one seek in flight; if the video is seeking, store the newest target and apply it after seek completion, discarding stale intermediate targets. This prevents a backlog of seeks and reduces seek flooding.
- Skip redundant seeks when the target is effectively the current/pending time; use a small configurable threshold while preserving accurate endpoint behavior.
- Keep the video and HTML elements driven by the same normalized progress. If any visual interpolation is used to reduce jitter, keep it bounded and responsive, do not introduce visible lag, and do not let it continue animating when input is idle.
- Use requestVideoFrameCallback when supported only for optional frame-aware visual synchronization; provide a safe fallback and do not make it a requirement for the scrubber to work.
- Do not use setInterval, continuous polling, or a permanent animation loop just to check video time.

Asset and build efficiency:
- Keep components small and reusable; avoid unnecessary dependencies and oversized libraries.
- Ensure the video is web-friendly (MP4/H.264 where possible, reasonable resolution/bitrate, fast-start metadata). Do not change or degrade the provided source unless necessary; if conversion is needed, explain the required format/settings rather than inventing a replacement asset.
- Avoid rendering large static elements repeatedly; keep decorative layers lightweight.
- Verify that the page remains responsive while scrubbing and that the CPU/GPU workload is not needlessly increased.

SHARED ANIMATION PROGRESS

Create one normalized progress value from 0 to 1. This exact same progress must control:
1. video.currentTime = progress × video.duration
2. the position, rotation, and scale of every floating HTML/CSS element

Do not create separate timelines or independently timed animations. No autoplay, automatic easing loop, or independent GSAP timeline should move the scene when the pointer is still. If GSAP is used for presentation, it must be driven by this shared progress and must not create autonomous playback.

FLOATING ELEMENTS

Create the following as real HTML/CSS elements, not baked into an image:
- Fun!
- Game/controller icon
- Explore!
- Smiley
- Adventure!
- Collect!

Match the references with rounded organic pills, playful but refined typography, pastel colors, and a few small hand-drawn decorative marks. Give each element its own defined START and END position, rotation, and scale. Interpolate each property from start to end using the shared progress. Elements must visibly travel outward from the compact first state to the expanded second state; do not merely fade them in/out or move them randomly. Their movement should feel coordinated with Shaun's rise. Keep spacing clean and prevent overlap with the headline, navigation, and Shaun's focal area.

LAYOUT & VISUAL DIRECTION

Recreate the composition using real responsive HTML/CSS:

Background: supplied Shaun video, full viewport, fixed behind UI, object-fit: cover.
Header: transparent Shaun logo top-left; Home / Game / Characters / About / Contact centered; yellow "Play Game" CTA with supplied controller icon top-right.
Hero heading: stable and centered, reading "IT'S TIME TO", "PLAY WITH", and "SHAUN THE SHEEP". Use the yellow rounded highlight around "PLAY" as shown in the references.
Composition: Shaun/video remain dominant. Keep the heading stable while Shaun and pills move. Match the reference spacing, scale, colors, and visual hierarchy without making the design cluttered, overly childish, or visually noisy. Use clear z-index layers so video, decorative elements, text, and controls never unintentionally obscure one another.

RESPONSIVE & TOUCH

Desktop: full mouse/pointer-controlled interaction using absolute vertical position.
Tablet: proportionally scale and reposition the composition while maintaining hierarchy and safe spacing.
Mobile: create a deliberate mobile layout, not a shrunken desktop; use a hamburger navigation and keep text readable.
Touch: use vertical swipe as the equivalent control. Swipe up increases progress; swipe down decreases it. Because touch has no hover position, map cumulative swipe distance to a bounded 0–1 progress using a configurable sensitivity. Stop changing progress when the touch ends. Prevent the browser's page-scroll gesture from hijacking the active hero interaction where appropriate, while preserving accessible page behavior. Do not autoplay on touch devices.

IMPLEMENTATION STRUCTURE

Use clean reusable React components, for example:
- Hero
- BackgroundVideo
- Header
- HeroHeading
- FloatingElements
- InteractionController

Centralize video readiness, pointer/touch input, normalized progress, seek scheduling, and cleanup. Use refs for high-frequency values and React state only for low-frequency UI states such as loading/error or menu open/closed. Clean up all event listeners, observers, and scheduled animation frames on unmount. Avoid unnecessary libraries; use GSAP only if it materially improves the result without creating a second independent timeline.

TESTING & SUCCESS CRITERIA

Before finishing, test and correct the following:
- Video asset loads from the correct Vite path and appears behind the UI.
- Video does not autoplay, loop, or play through video.play().
- Initial state is time 0 / progress 0 / collapsed composition.
- Pointer at bottom reaches 0%; pointer at top/header reaches 100% of the actual loaded duration (expected 6 seconds).
- Intermediate pointer positions map proportionally to progress and time.
- Pointer moving up advances; moving down reverses; stopping freezes the frame and all elements.
- Fast pointer movement does not create a backlog of seeks, visible freezes, or excessive React rerenders.
- Elements remain synchronized with the same progress and return precisely to their start positions.
- Mobile swipe works in both directions without autoplay.
- Layout works at common desktop, tablet, and mobile sizes without overlap or clipped essential content.
- No console errors, runaway animation loops, duplicated listeners, or memory leaks.

The final experience should feel like one continuous, responsive, pointer-controlled animation: the user moves the pointer, Shaun’s video scrubs in sync, the pills spread or collapse, and the entire scene holds perfectly when input stops. Preserve the design and behavior while prioritizing responsiveness, efficiency, and reliable playback scrubbing.

FINAL INSTRUCTION

Inspect the project and supplied assets before implementation. Build the complete working responsive hero, validate the video source and interaction, and optimize performance from the start. Do not omit the actual video, replace it with a still image, or alter the intended interaction to make implementation easier. If a requirement cannot be completed because an asset is unavailable or a browser limitation exists, state the exact limitation and provide the closest reliable implementation without pretending it is solved.